

AGEX API Specification

1. 문서 개요

1.1 문서 목적

본 문서는 AGEX의 공식 Public API 구조, Resource API, Execution API, Event API, Management API, Authentication, Authorization, Versioning, Error Contract, Pagination, Idempotency, Streaming, Webhook 및 Compatibility 기준을 정의한다.

AGEX API는 단순한 외부 호출 Interface가 아니다.

AGEX의 Application Layer, SDK, CLI, Console, 외부 시스템 및 Extension은 본 문서에서 정의하는 공식 API Contract를 통해 Platform 기능에 접근해야 한다.

본 문서는 다음 설계의 기준으로 사용한다.

- API Architecture
- API Endpoint Structure
- API Resource Model
- Authentication
- Authorization
- Tenant Context
- Resource API
- Agent API
- Workflow API
- Knowledge API
- Memory API
- Plugin API
- Marketplace API
- Execution API
- Event API
- Audit API

- Usage API
- Administration API
- Streaming API
- Webhook API
- Error Contract
- Pagination
- Filtering
- Sorting
- Idempotency
- Concurrency Control
- Versioning
- Deprecation
- Rate Limit
- API Observability
- API Security

1.2 적용 범위

본 문서는 AGEX의 외부 및 공개 내부 API Contract를 정의한다.

다음 세부 영역은 관련 문서와 연계한다.

- Product Architecture
- Core Architecture
- Runtime Architecture
- Agent Framework
- Workflow Engine
- Knowledge Engine
- Memory Engine

- Plugin Framework
- Marketplace
- SDK
- Security Architecture
- Multi-Tenant Architecture
- Governance

1.3 API의 공식 정의

AGEX API는 AGEX Resource와 Runtime 기능을 외부 Client가 공식적으로 사용하기 위한 Versioned Contract이다.

API는 최소한 다음 원칙을 따라야 한다.

- Resource Oriented
- Tenant Aware
- Authenticated
- Authorized
- Versioned
- Schema Validated
- Idempotent Where Required
- Observable
- Auditable
- Backward Compatible Where Defined

2. API 설계 목표

AGEX API는 다음 목표를 충족해야 한다.

2.1 일관성

모든 Resource API는 동일한 Naming, Error, Pagination 및 Version 규칙을 사용해야 한다.

2.2 확장성

새로운 Resource와 Capability를 기존 API 구조를 깨뜨리지 않고 추가할 수 있어야 한다.

2.3 보안성

모든 요청은 Identity, Tenant, Permission 및 Policy를 검증해야 한다.

2.4 명시성

Input, Output, Error 및 State Transition이 Schema로 정의되어야 한다.

2.5 SDK 친화성

API Contract에서 SDK와 타입을 생성할 수 있어야 한다.

2.6 자동화 가능성

모든 핵심 관리 기능은 UI 없이 API만으로 수행 가능해야 한다.

2.7 장기 호환성

API Version 정책과 Deprecation 절차를 통해 기존 Client의 안정성을 보장해야 한다.

2.8 추적성

모든 요청을 Request ID, Correlation ID 및 Trace ID로 추적할 수 있어야 한다.

3. API 핵심 원칙

AGEX API는 다음 원칙을 따른다.

- API First
- Contract First
- Schema First
- Resource Oriented
- Explicit Versioning
- Secure by Default
- Default Deny
- Tenant Context Required

- Stable Identifier
 - Immutable Published Version
 - Idempotent Mutation
 - Optimistic Concurrency
 - Standard Error Contract
 - Cursor Pagination
 - Streaming for Long Output
 - Async for Long Execution
 - No Internal Database Exposure
 - No Hidden Side Effect
 - Observable by Default
 - Auditable Sensitive Action
-

4. API 전체 구조

AGEX API는 다음 주요 범주로 구성한다.

1. Identity API
2. Tenant API
3. Resource API
4. Agent API
5. Workflow API
6. Knowledge API
7. Memory API
8. Plugin API
9. Model API
10. Marketplace API

- 11. Execution API
- 12. Event API
- 13. Artifact API
- 14. Policy API
- 15. Audit API
- 16. Usage API
- 17. Administration API
- 18. Webhook API
- 19. Health API

5. Base URL 구조

5.1 기본 구조

개념적인 API Base URL은 다음 형식을 따른다.

`https://{host}/api/{version}`

예:

`/api/v1`

5.2 Tenant 식별

Tenant는 다음 방식 중 하나로 전달할 수 있다.

- Access Token Claim
- Request Header
- Resource Path
- 명시적 Tenant Context

일반 Client에서 동일 Tenant 정보가 여러 위치에 상충하는 경우 요청을 거부해야 한다.

5.3 Resource URI

Resource URI는 복수형 명사를 기본으로 한다.

예:

/agents

/workflows

/knowledge-collections

/memories

/plugins

/executions

5.4 Action Endpoint

Resource Lifecycle 또는 복잡한 상태 변경은 명시적 Action Endpoint를 사용할 수 있다.

예:

POST /agents/{agentId}/publish

POST /executions/{executionId}/cancel

POST /workflows/{workflowId}/activate

단순 상태 변경을 임의의 RPC 형태로 남발하지 않는다.

6. API Protocol

6.1 기본 Protocol

Public API는 HTTPS를 기본으로 한다.

6.2 Payload Format

기본 데이터 교환 형식은 JSON을 사용한다.

6.3 추가 형식

필요한 경우 다음을 지원할 수 있다.

- Multipart
- Binary Stream
- Server-Sent Events

- WebSocket
- Signed Webhook

6.4 Character Encoding

UTF-8을 기본 인코딩으로 사용한다.

6.5 Date and Time

시간 값은 Offset 또는 UTC가 포함된 ISO 8601 형식을 사용한다.

예:

2026-08-21T10:00:00+09:00

7. API Resource 공통 구조

7.1 Resource Envelope

일반 Resource 응답은 가능한 경우 다음 공통 필드를 가진다.

```
{  
  "id": "resource-id",  
  "type": "agent",  
  "tenant_id": "tenant-id",  
  "name": "resource-name",  
  "version": "1.0.0",  
  "revision": 4,  
  "state": "active",  
  "created_at": "2026-08-21T01:00:00Z",  
  "updated_at": "2026-08-21T01:10:00Z",  
  "metadata": {}  
}
```

7.2 Resource ID

Resource ID는 생성 후 변경하지 않는다.

7.3 External Name과 ID

사람이 읽는 Name과 시스템 식별 ID를 구분한다.

7.4 Version

Versioned Resource는 명시적 Version을 가진다.

7.5 Revision

동일 Version의 수정 가능한 상태에는 Revision을 사용할 수 있다.

8. Authentication

8.1 지원 가능한 인증 방식

AGEX API는 배포 형태에 따라 다음 인증 방식을 지원할 수 있다.

- OAuth 2.x
- OpenID Connect
- API Credential
- Service Identity
- Workload Identity
- Signed Service Token
- Mutual TLS

8.2 Bearer Token

일반 사용자 및 Service 요청은 Bearer Token을 사용할 수 있다.

8.3 Token Claim

Token은 가능한 경우 다음 정보를 포함하거나 조회 가능해야 한다.

- Subject
- Tenant
- Roles

- Scopes
- Issuer
- Audience
- Issued At
- Expiration

8.4 Credential 직접 전달 금지

사용자 비밀번호나 Provider Credential을 일반 API 요청에 반복 전달하지 않는다.

8.5 만료 Token

만료된 Token은 요청을 거부한다.

9. Authorization

9.1 평가 요소

Authorization은 다음을 평가한다.

- Principal
- Tenant
- Action
- Resource
- Scope
- Role
- Attributes
- Policy
- Environment

9.2 권한 표현

개념적으로 다음과 같은 Permission 구조를 사용할 수 있다.

agent.read

agent.create
agent.update
agent.execute
agent.publish

workflow.read
workflow.execute

plugin.install
plugin.execute

9.3 API Endpoint와 Permission

모든 보호 Endpoint는 요구 Permission을 명시해야 한다.

9.4 Resource-Level Authorization

동일 Resource Type이라도 Resource Scope에 따라 결과를 제한해야 한다.

9.5 관리자 API

관리자 Endpoint도 Authentication, Authorization 및 Audit를 우회하지 않는다.

10. Tenant Context

10.1 Tenant 필수

Tenant Resource 요청은 Tenant Context가 필수이다.

10.2 Tenant 불일치

Token의 Tenant와 요청 Tenant가 일치하지 않으면 기본적으로 거부한다.

10.3 Platform Scope

Platform 관리자 기능은 별도 Platform Scope를 사용한다.

10.4 Tenant 누락

Tenant Resource 요청에서 Tenant를 결정할 수 없으면 요청을 실행하지 않는다.

11. Request 공통 Header

AGEX API는 다음 공통 Header를 사용할 수 있다.

Authorization

Content-Type

Accept

X-Request-ID

X-Correlation-ID

Idempotency-Key

If-Match

X-AGEX-Tenant

X-AGEX-API-Version

11.1 Request ID

Client가 제공하지 않으면 Server가 생성한다.

11.2 Correlation ID

복수 요청 또는 분산 실행을 연결하는 데 사용한다.

11.3 Trace Context

표준 분산 Trace Header를 지원할 수 있다.

12. Response 공통 Header

응답에는 가능한 경우 다음 정보를 제공한다.

X-Request-ID

X-Correlation-ID

X-RateLimit-Limit

X-RateLimit-Remaining

X-RateLimit-Reset

ETag

Retry-After

민감한 내부 Service 또는 Infrastructure 정보는 Header에 노출하지 않는다.

13. HTTP Method 규칙

13.1 GET

Resource 조회에 사용한다.

상태를 변경하지 않는다.

13.2 POST

Resource 생성, Execution 생성 또는 Action에 사용한다.

13.3 PUT

Resource 전체 교체가 명확한 경우 사용한다.

13.4 PATCH

부분 수정에 사용한다.

13.5 DELETE

삭제 또는 삭제 요청에 사용한다.

13.6 안전성

GET 요청이 외부 Side Effect를 발생시키도록 구현하지 않는다.

14. Status Code 기준

AGEX는 표준 HTTP Status Code를 기본으로 사용한다.

14.1 200 OK

정상 조회 또는 동기 작업 성공.

14.2 201 Created

Resource가 생성됨.

14.3 202 Accepted

비동기 작업이 접수됨.

14.4 204 No Content

본문 없이 성공.

14.5 400 Bad Request

요청 형식 또는 기본 Validation 실패.

14.6 401 Unauthorized

인증되지 않음.

14.7 403 Forbidden

인증은 되었으나 권한 없음.

14.8 404 Not Found

Resource를 찾을 수 없음.

보안상 존재 여부를 노출하면 안 되는 경우에도 사용할 수 있다.

14.9 409 Conflict

Version, Revision, State 또는 중복 충돌.

14.10 412 Precondition Failed

ETag 또는 조건부 요청 실패.

14.11 422 Unprocessable Entity

Schema는 유효하지만 Domain Rule을 충족하지 못함.

14.12 429 Too Many Requests

Rate Limit 또는 일부 Quota 초과.

14.13 500 Internal Server Error

예상하지 못한 Server 오류.

14.14 502 / 503 / 504

외부 Provider 또는 Service 가용성 문제에 따라 사용할 수 있다.

15. Error Contract

15.1 표준 구조

모든 오류는 가능한 한 다음 형식을 따른다.

```
{  
  "error": {  
    "code": "AGEX_RESOURCE_VERSION_CONFLICT",  
    "category": "conflict",  
    "message": "The requested resource revision is no longer current.",  
    "retryable": false,  
    "request_id": "req_xxx",  
    "correlation_id": "corr_xxx",  
    "details": {  
      "resource_id": "agent_xxx",  
      "current_revision": 8  
    },  
    "timestamp": "2026-08-21T01:10:00Z"  
  }  
}
```

15.2 Error Code

Error Code는 HTTP Status보다 구체적인 Platform 의미를 제공한다.

15.3 안정성

공개 Error Code는 API Contract의 일부로 관리한다.

15.4 Stack Trace

Stack Trace와 내부 경로를 외부 응답에 노출하지 않는다.

15.5 Secret 제거

Error Message와 Details에 Secret을 포함하지 않는다.

16. Error Category

표준 Category 예시는 다음과 같다.

- authentication
- authorization
- validation
- policy
- not_found
- conflict
- quota
- rate_limit
- timeout
- provider
- dependency
- runtime
- internal

16.1 Retryable

Client가 안전하게 재시도 가능한지 별도로 표시할 수 있다.

16.2 Suggested Action

필요한 경우 사람이 이해할 수 있는 해결 정보를 제공할 수 있다.

17. Idempotency

17.1 적용 대상

중복 호출 시 Side Effect가 발생할 수 있는 POST 요청은 Idempotency를 지원해야 한다.

대표 대상:

- Execution 생성
- Package 설치
- 승인 처리
- 외부 Side Effect가 있는 Action

17.2 Idempotency-Key

Client는 다음 Header를 사용할 수 있다.

Idempotency-Key: <unique-key>

17.3 동일 Key

동일 Key와 동일 Request에 대해서는 기존 결과를 반환할 수 있다.

17.4 Payload 불일치

같은 Key에 다른 Payload가 사용되면 Conflict로 처리한다.

17.5 보존 기간

Idempotency Record 보존 기간은 Operation 유형별로 정의한다.

18. Optimistic Concurrency Control

18.1 Revision 또는 ETag

Resource 수정 시 현재 Revision 또는 ETag를 사용할 수 있다.

18.2 If-Match

예:

If-Match: "revision-8"

18.3 충돌

다른 Client가 먼저 변경했으면 409 또는 412를 반환한다.

18.4 Lost Update 방지

중요 Resource의 PATCH가 마지막 Write Wins만으로 처리되지 않도록 한다.

19. Pagination

19.1 기본 방식

대량 Resource 조회는 Cursor Pagination을 기본으로 한다.

19.2 요청 예

GET /agents?limit=50&cursor=abc123

19.3 응답 예

```
{
  "items": [],
  "page": {
    "next_cursor": "next123",
    "has_more": true
  }
}
```

19.4 Offset Pagination

특정 관리 조회에서는 Offset Pagination을 사용할 수 있으나 대규모 Resource 기본 방식으로 사용하지 않는다.

19.5 Maximum Limit

Server는 최대 Page Size를 제한한다.

20. Filtering

20.1 표준 Filter

Resource별로 다음과 같은 Filter를 지원할 수 있다.

- state
- name
- type
- version
- owner
- created_after
- updated_after
- labels
- capability

20.2 Tenant Filter

Tenant Filter는 Client 일반 Query 조건이 아니라 보안 Scope로 적용한다.

20.3 임의 Query Language

초기 Public API에서 제한 없는 Query Language를 노출하지 않는다.

20.4 Filter Validation

지원되지 않는 Filter는 조용히 무시하지 않고 오류 또는 명확한 경고를 제공해야 한다.

21. Sorting

예:

sort=created_at:desc

21.1 지원 필드

각 Endpoint는 허용 Sorting Field를 명시한다.

21.2 안정적 정렬

Pagination과 함께 사용하는 정렬은 Stable Ordering을 제공해야 한다.

22. Field Selection

대규모 Resource 응답에서 필요 시 Field Projection을 지원할 수 있다.

예:

fields=id,name,state,version

22.1 보안

Field Selection으로 원래 권한이 없는 필드를 조회할 수 없어야 한다.

23. Expansion

연관 Resource를 제한적으로 함께 반환하는 Expansion을 지원할 수 있다.

예:

expand=owner,dependencies

23.1 제한

무제한 Nested Expansion을 허용하지 않는다.

23.2 비용

고비용 Expansion은 제한하거나 별도 Endpoint를 사용할 수 있다.

24. Identity API

24.1 목적

현재 Principal과 인증 Context를 조회한다.

24.2 예시 Endpoint

GET /identity/me

GET /identity/me/permissions

GET /identity/me/tenants

24.3 민감 정보

Token Raw Value 또는 Credential을 반환하지 않는다.

25. Tenant API

25.1 주요 Endpoint

POST /tenants

GET /tenants/{tenantId}

PATCH /tenants/{tenantId}

GET /tenants/{tenantId}/usage

GET /tenants/{tenantId}/quotas

PATCH /tenants/{tenantId}/quotas

25.2 Tenant 생성

Platform Scope 권한이 필요하다.

25.3 Tenant 상태

예:

- active
- suspended
- disabled
- deleting

25.4 Tenant 삭제

즉시 물리 삭제가 아닌 관리형 Lifecycle로 수행한다.

26. Agent API

26.1 Resource Endpoint

POST /agents

GET /agents

GET /agents/{agentId}

PATCH /agents/{agentId}

DELETE /agents/{agentId}

26.2 Version Endpoint

GET /agents/{agentId}/versions

GET /agents/{agentId}/versions/{version}

POST /agents/{agentId}/versions

26.3 Lifecycle Endpoint

POST /agents/{agentId}/validate

POST /agents/{agentId}/publish

POST /agents/{agentId}/activate

POST /agents/{agentId}/suspend

POST /agents/{agentId}/deprecate

26.4 Execution

POST /agents/{agentId}/executions

26.5 Agent Execution Request

```
{  
  "version": "1.2.0",  
  "input": {},  
  "context": {},  
  "execution": {  
    "priority": "normal",  
    "timeout_seconds": 300,  
    "budget": {}  
  }  
}
```

26.6 Agent 실행 응답

짧은 대기 실행을 제외하면 Execution Resource를 반환한다.

27. Workflow API

27.1 Resource Endpoint

POST /workflows

GET /workflows

GET /workflows/{workflowId}

PATCH /workflows/{workflowId}

DELETE /workflows/{workflowId}

27.2 Lifecycle

POST /workflows/{workflowId}/validate

POST /workflows/{workflowId}/publish

POST /workflows/{workflowId}/activate

POST /workflows/{workflowId}/suspend

27.3 Execution

POST /workflows/{workflowId}/executions

27.4 Instance 조회

GET /workflow-instances/{instanceId}

27.5 Instance Control

POST /workflow-instances/{instanceId}/pause

POST /workflow-instances/{instanceId}/resume

POST /workflow-instances/{instanceId}/cancel

28. Knowledge API

28.1 Source Endpoint

POST /knowledge-sources

GET /knowledge-sources

GET /knowledge-sources/{sourceId}

PATCH /knowledge-sources/{sourceId}

DELETE /knowledge-sources/{sourceId}

28.2 Collection Endpoint

POST /knowledge-collections

GET /knowledge-collections

GET /knowledge-collections/{collectionId}

28.3 Ingestion

POST /knowledge-sources/{sourceId}/ingestions

GET /knowledge-ingestions/{ingestionId}

28.4 Retrieval

POST /knowledge/retrieve

요청 예:

```
{  
  "query": "query text",  
  "collections": ["collection_x"],  
  "top_k": 10,  
  "filters": {},  
  "citation": true,  
  "freshness": {}  
}
```

28.5 Retrieval 응답


```
{
  "query_id": "qry_x",
  "results": [
    {
      "knowledge_id": "kn_x",
      "content": "...",
      "score": 0.94,
      "source": {},
      "citation": {},
      "metadata": {}
    }
  ]
}
```

29. Memory API

29.1 Search

POST /memories/search

29.2 Memory Proposal

POST /memories/proposals

29.3 조회

GET /memories/{memoryId}

29.4 수정

PATCH /memories/{memoryId}

권한 및 Memory Type에 따라 제한된다.

29.5 삭제

DELETE /memories/{memoryId}

29.6 User Memory 관리

GET /users/{userId}/memories

DELETE /users/{userId}/memories

전체 삭제는 별도 높은 권한과 Audit를 요구할 수 있다.

30. Plugin API

30.1 Registry

GET /plugins

GET /plugins/{pluginId}

GET /plugins/{pluginId}/versions

30.2 Installation

POST /plugin-installations

GET /plugin-installations

GET /plugin-installations/{installationId}

PATCH /plugin-installations/{installationId}

DELETE /plugin-installations/{installationId}

30.3 Action 실행

POST /plugin-installations/{installationId}/actions/{actionId}

30.4 직접 Provider URL 금지

Plugin Action API가 외부 Endpoint나 Credential을 Client에 노출하지 않는다.

31. Model API

31.1 Model Registry

GET /models

GET /models/{modelId}

GET /model-providers

31.2 Model Profile

POST /model-profiles

GET /model-profiles/{profileId}

PATCH /model-profiles/{profileId}

31.3 Direct Model Execution

권한과 정책에 따라 다음과 같은 API를 제공할 수 있다.

POST /model-executions

31.4 Provider 추상화

일반 Client가 Provider 전용 API를 직접 호출하도록 Public Contract를 설계하지 않는다.

32. Marketplace API

32.1 Catalog

GET /marketplace/packages

GET /marketplace/packages/{packageId}

32.2 Search

GET /marketplace/packages?query=...&type=plugin

32.3 Install

POST /marketplace/installations

32.4 Upgrade

POST /marketplace/installations/{installationId}/upgrade

32.5 Rollback

POST /marketplace/installations/{installationId}/rollback

32.6 Publisher

POST /marketplace/submissions

GET /marketplace/submissions/{submissionId}

33. Execution API

33.1 Execution Resource

Execution은 Agent, Workflow, Plugin, Model 및 System Task에 공통으로 사용할 수 있는 최상위 실행 Resource이다.

33.2 조회

GET /executions/{executionId}

33.3 목록

GET /executions

33.4 취소

POST /executions/{executionId}/cancel

33.5 Pause

지원되는 Execution은 다음을 제공할 수 있다.

POST /executions/{executionId}/pause

POST /executions/{executionId}/resume

33.6 강제 종료

POST /executions/{executionId}/terminate

높은 권한과 Audit가 필요하다.

34. Execution Resource 구조

예:

```
{  
  "id": "exec_x",
```

```
"type": "agent",
"tenant_id": "tenant_x",
"target": {
  "resource_id": "agent_x",
  "version": "1.0.0"
},
"state": "running",
"progress": {
  "current_step": "step_x"
},
"created_at": "...",
"started_at": "...",
"deadline": "...",
"usage": {},
"links": {
  "events": "/executions/exec_x/events",
  "result": "/executions/exec_x/result"
}
}
```

34.1 Execution Snapshot

일반 Client에 모든 내부 Snapshot을 반환하지 않는다.

조회 가능한 공개 Metadata만 제공한다.

35. Execution Result API

GET /executions/{executionId}/result

35.1 완료 전

결과가 아직 준비되지 않았다면 현재 상태를 반환하거나 적절한 Conflict/Accepted 상태를 제공한다.

35.2 Partial Result

지원되는 Execution은 Partial Result 여부를 명확히 표시해야 한다.

35.3 Artifact

대용량 결과는 Artifact Reference를 반환할 수 있다.

36. Execution Event API

GET /executions/{executionId}/events

36.1 Event 예

- execution.created
- execution.started
- task.started
- task.completed
- execution.waiting
- execution.completed
- execution.failed

36.2 내부 정보 제한

내부 Chain-of-Thought 또는 민감한 Runtime Detail은 Public Event에 포함하지 않는다.

37. Streaming API

37.1 지원 방식

Streaming은 다음 방식으로 제공할 수 있다.

- Server-Sent Events

- WebSocket
- Chunked Response

37.2 기본 Event 구조

```
{  
  "event": "output.delta",  
  "execution_id": "exec_x",  
  "sequence": 12,  
  "data": {}  
}
```

37.3 Streaming Event 유형

- execution.started
- output.delta
- output.item
- tool.requested
- status.changed
- usage.updated
- execution.completed
- execution.failed

37.4 Stream과 Execution State

Stream이 끊겼다고 Execution이 종료된 것은 아니다.

최종 상태는 Execution API에서 확인한다.

37.5 Resume

가능한 경우 Sequence 또는 Cursor 기반 재연결을 지원한다.

38. Event API

38.1 Event 발행

POST /events

38.2 Event Subscription

POST /event-subscriptions

GET /event-subscriptions

DELETE /event-subscriptions/{subscriptionId}

38.3 Event Schema

모든 Event Type은 Schema Version을 가진다.

38.4 Event 권한

Client는 허용된 Event Type만 발행할 수 있다.

38.5 Event ID

중복 처리 방지를 위해 Event ID를 사용한다.

39. Event 요청 구조

예:

```
{  
  "type": "custom.resource.changed",  
  "version": "1",  
  "subject": "resource_x",  
  "data": {},  
  "correlation_id": "corr_x"  
}
```

39.1 System Reserved Event

AGEX 시스템 Event Namespace는 일반 Client가 임의 발행할 수 없도록 제한한다.

40. Webhook API

40.1 목적

AGEX Event 또는 Execution 상태를 외부 시스템으로 전달할 수 있다.

40.2 Webhook 등록

POST /webhooks

GET /webhooks

PATCH /webhooks/{webhookId}

DELETE /webhooks/{webhookId}

40.3 Webhook 설정

- Endpoint
- Events
- Secret Reference
- Retry
- Timeout
- Enabled

40.4 Delivery

Webhook 전달은 Delivery ID를 가져야 한다.

40.5 Signature

Webhook Payload는 서명할 수 있어야 한다.

40.6 Retry

일시적인 실패는 제한적으로 재시도한다.

41. Webhook Delivery API

GET /webhooks/{webhookId}/deliveries

GET /webhook-deliveries/{deliveryId}

POST /webhook-deliveries/{deliveryId}/retry

수동 Retry에는 권한과 Audit를 적용한다.

42. Artifact API

42.1 목적

대용량 Input, Output, 파일 및 Runtime Artifact를 관리한다.

42.2 Endpoint 예

POST /artifacts

GET /artifacts/{artifactId}

DELETE /artifacts/{artifactId}

42.3 Upload

대용량 Upload는 사전 서명된 제한 URL 또는 Streaming Upload 방식으로 처리할 수 있다.

42.4 Download

Artifact 접근도 Permission과 Tenant Scope를 검증한다.

42.5 만료

임시 Artifact는 Expiration을 가질 수 있다.

43. Policy API

43.1 Resource

POST /policies

GET /policies

GET /policies/{policyId}

PATCH /policies/{policyId}

43.2 Validation

POST /policies/{policyId}/validate

43.3 Simulation

실제 실행 없이 Policy Decision을 확인하는 API를 제공할 수 있다.

POST /policy-simulations

43.4 위험 제한

Policy Simulation이 실제 Permission을 부여하지 않는다.

44. Audit API

44.1 조회

GET /audit-records

GET /audit-records/{auditId}

44.2 Filter

- principal
- action
- resource
- execution
- time_range
- result

44.3 수정 금지

Public API에서 Audit Record 수정 또는 삭제 Endpoint를 제공하지 않는다.

44.4 민감한 Audit

Audit Record 조회 자체에도 별도 Permission이 필요하다.

45. Usage API

45.1 사용량 조회

GET /usage

GET /usage/summary

45.2 Filter

- period
- resource
- agent
- workflow
- plugin
- model
- application
- tenant

45.3 비용 정보

권한에 따라 Cost를 포함하거나 제외할 수 있다.

46. Administration API

46.1 범위

Platform 관리자 기능은 일반 Resource API와 별도 권한 Scope를 사용한다.

46.2 관리 대상

- Tenant
- Platform Configuration
- Global Quota
- Provider
- Trust Level
- Package Block
- Security Control

46.3 강력한 Action

고위험 관리자 Action에는 다음을 추가할 수 있다.

- Step-Up Authentication
 - Human Approval
 - Mandatory Reason
 - Immutable Audit
-

47. Health API

47.1 Public Health

외부 Load Balancer 등이 사용할 제한된 Health Endpoint를 제공할 수 있다.

GET /health

47.2 상세 Health

상세 Dependency 상태는 관리자 권한으로 제한한다.

GET /admin/health

47.3 정보 노출

Database Hostname, Secret, 내부 Network 구조 등을 Health 응답에 노출하지 않는다.

48. Asynchronous Operation Pattern

장시간 작업은 즉시 완료를 기다리지 않는다.

예:

POST /knowledge-sources/{id}/ingestions

응답:

```
{  
  "operation_id": "op_x",  
  "state": "accepted"
```

}

또는 Execution Resource를 반환한다.

48.1 Operation 조회

GET /operations/{operationId}

48.2 Operation과 Execution

실제 AI 또는 Runtime Task는 Execution으로 통일하고, 관리성 비동기 작업에 별도 Operation Resource를 사용할 수 있다.

49. Long-Running Operation

장기 작업은 다음 정보를 제공해야 한다.

- ID
- State
- Progress
- Created At
- Started At
- Completed At
- Error
- Result Reference

49.1 Client Polling

Polling Frequency를 과도하게 높이지 않도록 Retry-After를 제공할 수 있다.

49.2 Event 활용

완료 Event 또는 Webhook 사용을 지원한다.

50. Bulk API

50.1 제한적 지원

대량 생성·수정이 필요한 Resource에는 Bulk API를 제공할 수 있다.

50.2 예

POST /agents:bulk-import

실제 URI 규칙은 최종 표준에서 확정한다.

50.3 부분 성공

Bulk 작업은 Item별 결과를 반환해야 한다.

50.4 Transaction

모든 Bulk 요청이 전체 원자성을 보장한다고 가정하지 않는다.

50.5 최대 크기

Bulk 요청 크기를 제한한다.

51. Batch Request

서로 무관한 일반 API를 임의로 하나의 Batch Endpoint에서 실행하는 기능은 신중하게 제한한다.

51.1 이유

- Permission 복잡성
- Transaction 오해
- Audit 복잡성
- Rate Limit 우회

복잡한 실행 조합에는 Workflow를 사용하는 것을 우선한다.

52. Search API

복수 Resource에 대한 전역 검색이 필요한 경우 전용 Search API를 제공할 수 있다.

POST /search

52.1 검색 범위

Client가 접근 가능한 Resource만 반환한다.

52.2 Index

Search Index는 Source of Truth가 아니다.

53. Input Validation

모든 API Request는 최소한 다음을 검증한다.

- JSON Syntax
- Schema
- Required Fields
- Type
- Format
- Length
- Resource Reference
- Tenant Ownership
- Permission
- Policy

53.1 알 수 없는 Field

Endpoint 정책에 따라 거부하거나 무시할 수 있으나 일관되게 정의해야 한다.

고위험 Resource에서는 기본 거부를 권장한다.

53.2 최대 Payload

모든 Endpoint는 최대 Payload Size를 정의한다.

54. Output Validation

Server 내부 결과도 Public Schema에 맞게 검증해야 한다.

54.1 Provider Raw Response

외부 Provider 응답을 Public API Response로 그대로 노출하지 않는다.

54.2 내부 필드

다음은 일반적으로 Public API에서 제거한다.

- Secret
 - Internal Host
 - Database Key
 - Stack Trace
 - Private Runtime Metadata
-

55. API Schema Registry

55.1 정의

모든 Public Request/Response Schema는 중앙 Schema Registry 또는 Canonical Contract로 관리한다.

55.2 Schema ID

Schema는 ID와 Version을 가진다.

55.3 코드 생성

SDK 및 문서는 해당 Schema에서 생성할 수 있다.

55.4 변경 검증

Schema 변경 시 Compatibility Test를 수행한다.

56. API Versioning

56.1 Major API Version

Breaking Change는 Major API Version 증가를 요구한다.

예:

/api/v1

/api/v2

56.2 Minor 변경

Backward Compatible Field 추가는 동일 Major Version에서 가능하다.

56.3 Field 제거

공개 Field를 즉시 제거하지 않는다.

Deprecation 기간을 거친다.

56.4 의미 변경

같은 필드의 의미를 조용히 바꾸는 것을 금지한다.

57. Resource Version과 API Version 구분

다음은 서로 다른 개념이다.

API Version

Agent Version

Workflow Version

Plugin Version

Schema Version

Runtime Version

57.1 API Version

Client Contract의 Version이다.

57.2 Resource Version

Resource 자체 Definition의 Version이다.

57.3 혼동 금지

/api/v1/agents의 v1은 Agent Version을 의미하지 않는다.

58. Deprecation Policy

58.1 Deprecated 표시

다음 방식으로 안내할 수 있다.

- Documentation
- Response Header
- SDK Warning
- Developer Portal

58.2 제공 정보

- Deprecated Date
- Replacement
- End-of-Support Date
- Migration Guide

58.3 제거

지원 종료 전에 가능한 충분한 Migration 기간을 제공한다.

58.4 보안 예외

심각한 보안 취약점이 있는 API는 일반적인 Deprecation 기간보다 빠르게 제한될 수 있다.

59. Compatibility

59.1 Client Compatibility

신규 Server는 지원 기간 내의 이전 SDK 및 API Client와 호환되어야 한다.

59.2 Unknown Field

Client는 호환 정책에 따라 신규 응답 필드를 안전하게 무시할 수 있어야 한다.

59.3 Enum 확장

새 Enum 값 추가가 기존 Client 전체 실패로 이어지지 않도록 SDK를 설계할 수 있다.

60. Rate Limit

60.1 적용 범위

- Principal
- Tenant
- Endpoint
- Resource Type
- IP
- Execution Type

60.2 응답

Rate Limit 초과 시 429를 반환한다.

60.3 Header

가능하면 Limit, Remaining, Reset 정보를 제공한다.

60.4 Retry-After

재시도 가능한 시간을 제공할 수 있다.

60.5 우회 방지

여러 Credential을 통한 정책 우회를 탐지할 수 있어야 한다.

61. Quota

Rate Limit과 Quota는 구분한다.

61.1 Rate Limit

시간당 호출 속도를 제한한다.

61.2 Quota

누적 자원 사용량을 제한한다.

예:

- Storage

- Token
- Execution Count
- Concurrent Execution

61.3 오류

Quota 초과는 별도 Error Code로 명확히 구분한다.

62. Request Timeout

62.1 동기 API

모든 동기 API는 Server-side Timeout을 가진다.

62.2 장기 실행

Timeout보다 긴 작업은 비동기 Execution으로 전환한다.

62.3 Client Timeout

Client의 Connection Timeout과 Server Execution Timeout을 구분한다.

63. API Retry Semantics

63.1 GET

일시적 오류에 대해 일반적으로 재시도 가능하다.

63.2 POST

Idempotency가 확인되지 않은 상태 변경은 자동 재시도에 주의한다.

63.3 Retryable Error

Error Contract의 retryable 정보를 활용할 수 있다.

63.4 Server 권고

Retry-After가 제공되면 Client는 이를 우선해야 한다.

64. Web Security

64.1 TLS

모든 Public API는 암호화된 연결을 사용한다.

64.2 CORS

Browser Client의 Origin 정책을 명시적으로 설정한다.

64.3 CSRF

Cookie 기반 인증을 사용하는 Interface는 CSRF 방어를 적용한다.

64.4 Request Size

DoS 방지를 위해 Payload Size를 제한한다.

64.5 Content Type

예상하지 않은 Content Type은 거부한다.

65. API Security Boundary

API Gateway에서 최소한 다음을 적용한다.

Request

→ TLS

→ Threat Filtering

→ Authentication

→ Tenant Resolution

→ Rate Limit

→ Schema Validation

→ Authorization

→ Policy Evaluation

→ Domain Service

65.1 Domain 재검증

Gateway 검증만 신뢰하지 않는다.

중요 Domain Service에서도 Resource Permission과 Policy를 검증한다.

66. Data Classification in API

66.1 Request

민감한 Payload에는 Data Classification을 Context로 적용할 수 있다.

66.2 Response

Response에 포함되는 데이터의 Classification에 따라 Masking이나 제한을 적용한다.

66.3 Provider 호출

외부 Provider로 전달되는 경우 API 요청에서 실행된 Data Policy를 Runtime까지 전파해야 한다.

67. API Audit

다음 API Action은 기본 Audit 대상이다.

- Permission 변경
- Policy 변경
- Agent Publish
- Workflow Publish
- Plugin Install
- Secret Binding
- Tenant 변경
- Execution Terminate
- Memory Delete
- Package Block
- 관리자 Override

67.1 Read Audit

민감 데이터 조회는 정책에 따라 Audit할 수 있다.

68. API Observability

68.1 Metric

- Request Count
- Success Rate
- Error Rate
- Latency
- Payload Size
- Authentication Failure
- Authorization Failure
- Rate Limit
- Endpoint Usage

68.2 Trace

API 요청에서 시작된 Runtime Execution까지 동일 Correlation Context를 유지한다.

68.3 Log

구조화 Log를 사용한다.

68.4 Payload Logging

Request와 Response 전체 본문을 기본 Log에 기록하지 않는다.

69. Request Lifecycle

표준 API 요청은 다음 순서로 처리한다.

Request Received

→ Request ID

→ Authentication

- Tenant Resolution
 - API Version
 - Schema Validation
 - Rate Limit
 - Authorization
 - Policy Evaluation
 - Idempotency
 - Domain Operation
 - Audit
 - Usage
 - Response Validation
 - Response
-

70. Execution Request Lifecycle

API 실행 요청은 다음 순서로 처리한다.

Execution Request

- API Validation
- Authentication
- Tenant
- Permission
- Policy
- Resource Resolution
- Version Resolution
- Budget
- Quota

→ Core Execution Coordination

→ Runtime

→ Execution Resource Returned

API Layer가 Agent 또는 Model을 직접 실행하지 않는다.

71. API Gateway 역할

API Gateway는 다음 책임을 가진다.

- Routing
- TLS Termination
- Authentication Integration
- Basic Rate Limit
- Request Size Limit
- API Version Routing
- Request ID
- Threat Protection
- Access Logging

71.1 Gateway 비즈니스 로직

복잡한 Domain Logic은 Gateway에 구현하지 않는다.

71.2 Gateway State

가능한 한 Stateless하게 유지한다.

72. Internal Service API

AGEX 내부 Service 간 Interface도 명시적 Contract를 사용해야 한다.

72.1 Public과 Internal 구분

Internal API를 자동으로 Public API로 공개하지 않는다.

72.2 내부 인증

Service-to-Service 요청도 Service Identity를 사용한다.

72.3 Tenant

내부 요청에도 Tenant Context가 유지되어야 한다.

72.4 Direct Database Integration

Service 간 데이터 공유를 Database 직접 접근으로 구현하지 않는다.

73. API Scope와 SDK 관계

SDK는 Public API Contract 위에 구축한다.

Application

↓

SDK

↓

Public API

↓

Core / Runtime

SDK 전용 비공개 기능을 만들어 Public API와 기능 차이를 발생시키지 않는 것을 원칙으로 한다.

74. API와 UI 관계

AGEX Console과 Builder도 원칙적으로 동일 Public 또는 공식 Management API를 사용한다.

UI가 Core Database를 직접 조회하지 않는다.

이를 통해 다음을 보장한다.

- API/UI 기능 일치
- Automation 가능

- SDK 기능 일치
 - 권한 일관성
 - Audit 일관성
-

75. API와 Event 관계

API는 명령과 조회를 제공한다.

Event는 상태 변화를 비동기적으로 전달한다.

두 방식을 혼동하지 않는다.

예:

POST /executions

은 실행을 요청하고,

execution.completed

Event가 이후 완료 상태를 알릴 수 있다.

76. API와 Workflow 관계

복수 API Action을 Client에서 복잡하게 연결해야 하는 경우 Workflow로 승격하는 것을 고려한다.

API Client가 장시간 상태 기계를 직접 구현하도록 요구하지 않는 것을 원칙으로 한다.

77. API Documentation

77.1 공식 문서

Public API는 자동 생성 가능한 Reference를 제공해야 한다.

77.2 문서 내용

각 Endpoint는 최소한 다음을 포함한다.

- Purpose

- Method
- Path
- Authentication
- Permission
- Request Schema
- Response Schema
- Errors
- Rate Limit
- Example
- Version
- Deprecation

77.3 버전별 문서

API Version별 문서를 유지한다.

78. OpenAPI 및 Machine-Readable Contract

REST API를 사용하는 경우 OpenAPI와 같은 Machine-Readable Contract를 제공하는 것을 원칙으로 한다.

78.1 활용

- SDK Generation
- Validation
- Documentation
- Testing
- Compatibility Diff

78.2 Source of Truth

Contract Definition과 실제 Server 구현이 불일치하지 않도록 CI에서 검증해야 한다.

79. API Change Management

API 변경은 최소한 다음 절차를 따른다.

Change Proposal

- Contract Review
- Security Review
- Compatibility Analysis
- Schema Change
- Implementation
- Contract Test
- SDK Update
- Documentation
- Release

79.1 Breaking Change

Breaking Change는 별도 승인 절차를 요구한다.

80. API Contract Testing

80.1 대상

- Request Schema
- Response Schema
- Error Code
- HTTP Status
- Authentication
- Authorization
- Pagination

- Idempotency
- Versioning

80.2 Consumer Test

주요 SDK와 Client가 변경 API에서 정상 동작하는지 검증한다.

81. API Security Testing

다음 항목을 검증한다.

- Authentication Bypass
 - Authorization Bypass
 - Cross-Tenant Access
 - Object-Level Authorization
 - Mass Assignment
 - Injection
 - Oversized Payload
 - Rate Limit Bypass
 - Credential Leakage
 - Sensitive Error Exposure
 - Replay
 - Idempotency Abuse
-

82. API Load Testing

주요 Endpoint에 대해 다음을 검증한다.

- Throughput
- p50 Latency
- p95 Latency

- p99 Latency
- Error Rate
- Concurrent Request
- Rate Limit
- Queue Backpressure

82.1 AI 실행 API

Model 처리 시간을 API Gateway 성능과 혼동하지 않는다.

Execution 생성 Latency와 실제 Execution Duration을 별도로 측정한다.

83. API Availability

83.1 관리 API와 실행 API

필요한 경우 서로 다른 Scaling 및 Availability 정책을 적용할 수 있다.

83.2 Read API

일부 Read Model 장애 시 Source of Truth 또는 제한된 Fallback을 사용할 수 있다.

83.3 Write API

State Store를 확인할 수 없는 상황에서 성공으로 응답하지 않는다.

84. API Privacy

84.1 최소 데이터

API Response는 Client가 필요로 하는 최소 데이터를 기본으로 반환한다.

84.2 사용자 데이터

사용자 관련 데이터는 Tenant 및 Purpose Scope를 적용한다.

84.3 Export

대량 데이터 Export API는 별도 Permission, Audit 및 비동기 처리 방식을 사용할 수 있다.

85. API Governance

API Endpoint는 다음 Owner를 가져야 한다.

- Domain Owner
- Technical Owner
- Security Owner
- Schema Owner

85.1 Review 대상

- 신규 Endpoint
- Breaking Change
- Admin API
- Bulk API
- Sensitive Data API
- Cross-Service API

86. API Naming Convention

86.1 Resource

복수형 명사를 사용한다.

86.2 Field

JSON Field는 하나의 Naming Convention을 일관되게 사용한다.

본 문서 기준 Canonical JSON Field는 snake_case를 권장한다.

86.3 Boolean

의미가 명확한 이름을 사용한다.

예:

enabled

has_more

retryable

86.4 시간

다음과 같이 일관된 이름을 사용한다.

created_at

updated_at

started_at

completed_at

87. Identifier Convention

Resource ID는 Type을 식별하기 쉬운 Prefix를 사용할 수 있다.

예:

agt_

wfl_

plg_

exec_

mem_

kn_

pkg_

이는 선택적 구현 규칙이며 외부 Client가 Prefix Parsing에 의존해서는 안 된다.

88. Null과 Missing

88.1 구분

null과 Field 미포함의 의미를 Schema에서 명확히 정의해야 한다.

88.2 PATCH

PATCH에서 다음을 구분한다.

- Field 미포함: 변경 안 함
- Field = null: 값 제거 또는 null 설정

지원 여부는 Resource Schema로 정의한다.

89. Boolean Action 금지

다음과 같은 모호한 상태 API 사용을 피한다.

```
{  
  "published": true  
}
```

중요 Lifecycle 변경은 명시적 Action을 사용한다.

POST /agents/{id}/publish

이 방식은 권한, 검증 및 Audit 의미를 명확하게 한다.

90. Sensitive Administrative Actions

다음 작업은 일반 CRUD보다 강한 통제를 적용해야 한다.

- Tenant Delete
- Package Block
- Policy Override
- Secret Rotate
- Forced Execution Termination
- Bulk Memory Delete
- Permission Escalation

90.1 요구 가능 항목

- Reason

- Step-Up Authentication
 - Approval
 - Confirmation Token
 - Detailed Audit
-

91. API Multi-Region 고려사항

91.1 Region Endpoint

배포 방식에 따라 Region별 Endpoint 또는 Global Endpoint를 지원할 수 있다.

91.2 Data Residency

Tenant Data Policy에 따라 요청을 특정 Region으로 Routing해야 할 수 있다.

91.3 자동 Region 이동

정책을 위반하는 Region으로 자동 Fallback하지 않는다.

92. API Error Recovery

92.1 Client Action

Client가 오류 발생 후 수행해야 할 동작을 추론할 수 있도록 Error Contract를 제공한다.

92.2 Conflict

최신 Resource를 다시 조회 후 변경을 재적용하도록 할 수 있다.

92.3 Rate Limit

Retry-After를 따른다.

92.4 Authentication

Token Refresh 후 재시도할 수 있다.

92.5 Permission

자동으로 다른 Credential을 사용하여 우회하지 않는다.

93. API 테스트 환경

93.1 Test Endpoint

개발자는 Production과 분리된 Environment에서 API를 검증할 수 있어야 한다.

93.2 Test Tenant

독립적인 Test Tenant를 권장한다.

93.3 Mock Endpoint

Contract 개발을 위해 Mock Server를 제공할 수 있다.

93.4 Production 차이

Mock 성공이 실제 Permission 및 Policy 성공을 의미하지 않는다.

94. API Acceptance Criteria

Public API는 Release 전에 최소한 다음을 충족해야 한다.

1. Endpoint Owner가 정의되어 있다.
2. Request Schema가 존재한다.
3. Response Schema가 존재한다.
4. Error Contract가 정의되어 있다.
5. Authentication 요구사항이 명확하다.
6. Permission이 정의되어 있다.
7. Tenant Scope가 적용된다.
8. Rate Limit 정책이 존재한다.
9. Timeout이 존재한다.
10. Mutation의 Idempotency가 검토되었다.
11. Concurrency Control이 검토되었다.
12. Pagination이 필요한 경우 구현되었다.
13. API Version이 정의되어 있다.

14. Compatibility Test를 통과한다.
15. Security Test를 통과한다.
16. Tenant Isolation Test를 통과한다.
17. Audit 요구사항이 정의되어 있다.
18. Observability가 제공된다.
19. SDK Contract와 일치한다.
20. 공식 Documentation이 존재한다.

95. API 금지사항

다음 구조는 허용하지 않는다.

- Tenant Context 없이 Tenant Resource 접근
- 인증 없는 보호 Resource Endpoint
- Authorization 없는 Object 조회
- Core Database Table을 그대로 Public API에 노출
- 내부 Database ID 구조를 Business Contract로 강제
- Version 없는 Breaking Change
- Schema 없는 JSON Payload
- 모든 오류를 200 Response로 반환
- Stack Trace 외부 노출
- Secret 또는 Credential Response 포함
- GET 요청으로 상태 변경
- 무제한 Page Size
- 무제한 Request Payload
- Idempotency 없는 위험한 중복 POST
- Published Resource 직접 Update

- 장기 AI 실행을 HTTP Connection 하나에만 의존
 - Client 연결 종료를 Execution 취소로 간주
 - Permission 확대 Action의 Silent 처리
 - Audit 없는 관리자 강제 Action
 - API Gateway에 Domain Logic 집중
 - SDK만 접근 가능한 비공개 핵심 기능
 - UI에서 Database 직접 접근
-

96. API 검증 기준

새로운 API Endpoint 또는 Contract는 다음 질문을 통과해야 한다.

1. 어떤 Resource 또는 Domain에 속하는가.
2. Public API로 노출할 필요가 있는가.
3. URI와 HTTP Method가 의미에 맞는가.
4. Input Schema가 명확한가.
5. Output Schema가 명확한가.
6. Tenant Context가 유지되는가.
7. Authentication이 필요한가.
8. Permission이 정의되는가.
9. Policy 평가가 필요한가.
10. 상태 변경이면 Idempotency가 필요한가.
11. Concurrency 충돌 가능성이 있는가.
12. Error Code가 정의되는가.
13. 장기 작업이면 비동기 처리되는가.
14. Rate Limit이 필요한가.
15. Audit 대상인가.

16. Sensitive Data가 포함되는가.
 17. SDK에서 자연스럽게 표현 가능한가.
 18. 기존 Client와 호환되는가.
 19. Observability가 가능한가.
 20. AGEX Product Architecture와 Security 원칙에 부합하는가.
-

97. API Specification의 최종 정의

AGEX API는 Application Layer, SDK, CLI, Console, Extension 및 외부 시스템이 AGEX의 모든 공식 Resource와 Runtime 기능에 접근하기 위한 Versioned Public Contract이다.

AGEX API는 UI 구현을 위한 부가 기능이 아니라 Platform 기능의 공식 진입점이다.

Agent, Workflow, Knowledge, Memory, Plugin, Marketplace 및 Execution은 각각 명확한 Resource API를 가지며, 공통적으로 Tenant, Authentication, Authorization, Policy, Version, Schema 및 Audit 규칙을 적용한다.

장기 Agent와 Workflow 실행은 동기 HTTP 요청에 종속되지 않고 Execution Resource로 관리한다.

모든 상태 변경 API는 Idempotency와 동시성 문제를 고려해야 하며 Published Resource를 직접 변경하는 방식은 허용하지 않는다.

API Error는 HTTP Status와 AGEX Error Code를 조합해 명확히 표현하며 내부 Stack Trace나 Secret은 공개하지 않는다.

SDK와 Console은 동일한 공식 API Contract를 사용하며, SDK 전용 또는 UI 전용의 비공개 핵심 기능을 만들지 않는 것을 원칙으로 한다.

API 변경은 Schema, Compatibility, Security, SDK 및 Documentation을 함께 관리해야 하며 Breaking Change는 명시적 Version 변경과 Migration 절차를 요구한다.

AGEX API Specification은 다음 문장으로 최종 정의한다.

AGEX API는 AI Operating System의 모든 자원과 실행 능력을 인증·권한·정책·버전·스키마라는 명확한 계약 아래 외부 세계에 제공하는 AGEX의 공식 Programmatic Interface이다.

본 문서는 AGEX API의 Endpoint 구조, Resource Contract, 인증, 오류, 실행, Streaming, Event, Version 및 보안 원칙을 규정하는 공식 API Specification 문서로 사용한다.

